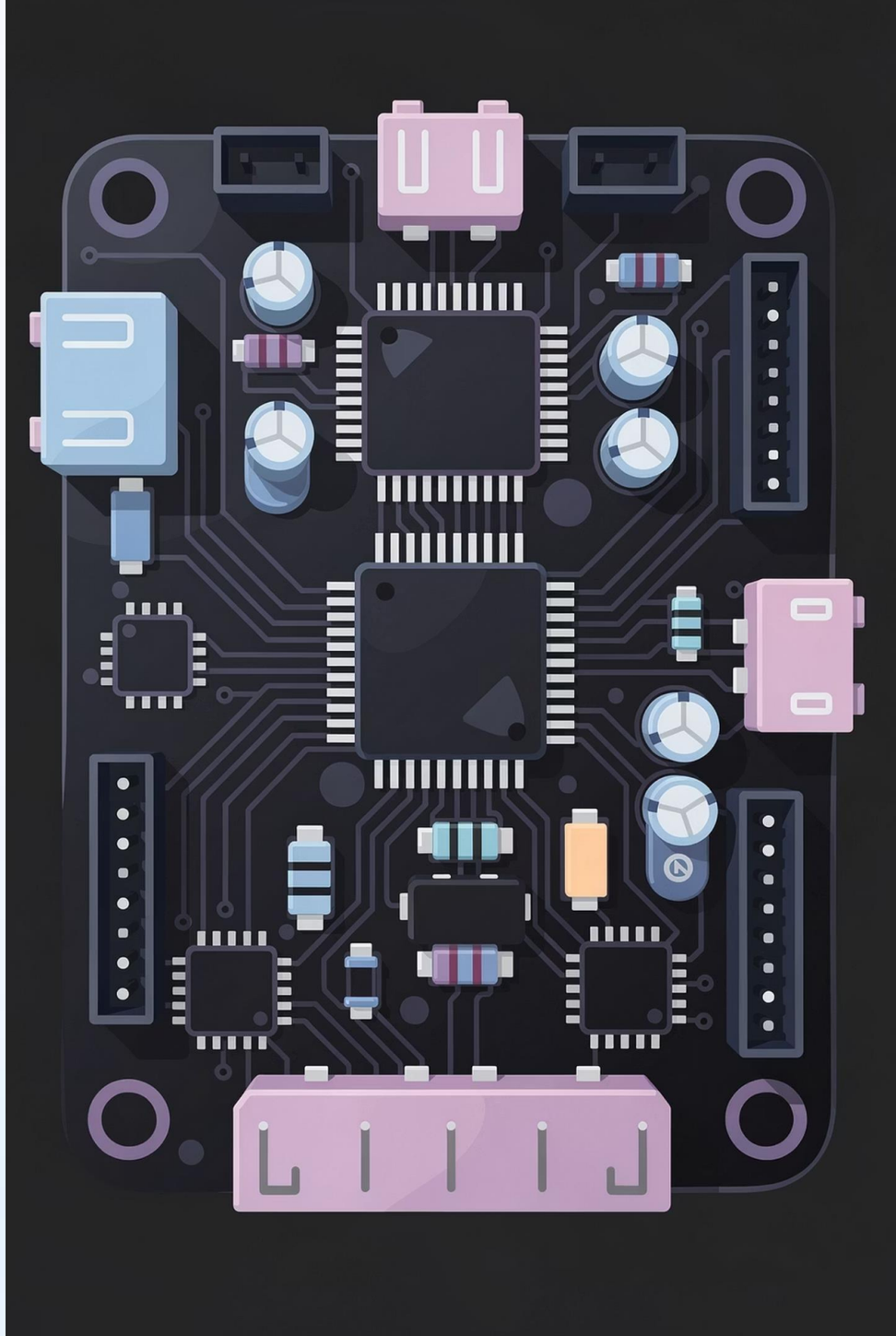
MODULE 03

Embedded Systems Fundamentals & Secure Boot Concepts

Course: Cybersecurity for Embedded & OT Systems

Focus: How embedded devices work and where security begins



Why This Module Matters

Embedded systems are the digital intelligence that directly sense, decide, and act on physical processes in industrial environments. Unlike traditional IT systems that process information, these devices bridge the gap between software logic and physical reality.

In operational technology environments, firmware behavior directly influences critical operational factors including equipment safety, process stability, and overall system availability. A software flaw in an embedded controller can translate into real-world consequences—from minor production delays to catastrophic equipment failure.

Security must begin inside the device, not only at the network perimeter. This fundamental principle distinguishes OT security from traditional IT approaches.

This module explains how embedded systems operate at a technical level and establishes the foundation for understanding how trust is established at device startup through secure boot mechanisms.

What Is an Embedded System (OT View)

An embedded system is a dedicated controller built to perform specific tasks reliably and predictably. Unlike general-purpose computers, these systems are purpose-built for singular functions within larger industrial processes.

Continuous Operation

Run 24/7 without reboots, often for years at a time

Real-Time Response

Execute commands within microsecond-level timing constraints

Harsh Environments

Withstand extreme temperatures, vibration, and electrical noise

Long Lifecycle

Expected to remain stable and functional for decades

Common Industrial Examples

PLC Control Modules

Programmable logic controllers managing automated processes

Motor Drives

Variable frequency drives controlling motor speed and torque

Protection Relays

Electrical protection devices monitoring grid conditions

Smart Sensors & HMIs

Intelligent sensors and human-machine interfaces

Industrial Microcontrollers (MCUs)

Industrial microcontrollers are the processing heart of embedded systems, designed specifically for predictability and reliability rather than raw computational performance. This design philosophy reflects the unique demands of operational technology environments.

Deterministic Timing

Execution times are predictable and repeatable, ensuring consistent response to inputs and events

Extended Lifecycle

Product availability guaranteed for 10–30 years to match industrial equipment lifespans

Integrated Peripherals

Built-in analog-to-digital converters (ADC), timers, and communication interfaces reduce external component count

Low Power, High Reliability

Optimized for minimal power consumption and maximum mean time between failures (MTBF)

Security Implication: Firmware running on an MCU directly defines system behavior. There is no operating system layer to provide abstraction or protection—the code has complete control over hardware and process interactions.

Basic MCU Architecture

Every microcontroller, regardless of manufacturer or complexity, contains three fundamental architectural components that work together to execute embedded applications.

CPU Core

The central processing unit executes instructions sequentially, performing arithmetic operations, logic comparisons, and control flow decisions.

Memory Systems

Flash Memory: Non-volatile storage containing firmware code and constant data, persists without power

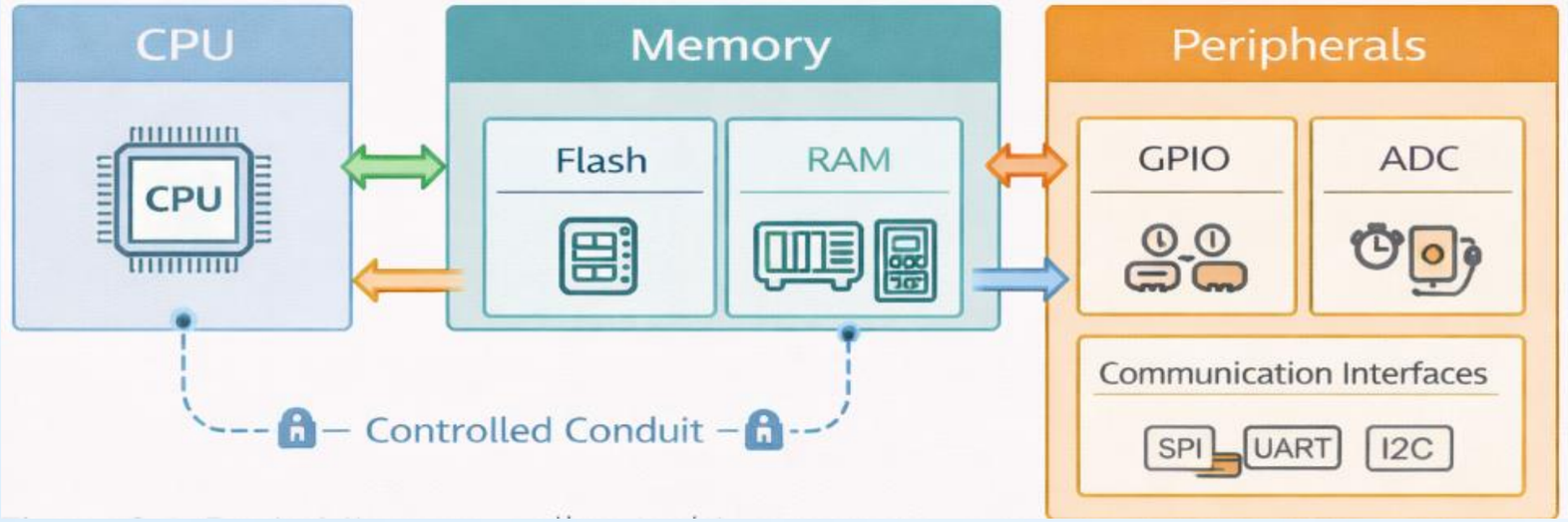
RAM: Volatile memory storing runtime variables, stack, and heap data during execution

Peripheral Interfaces

Timers: Generate precise time intervals and trigger events

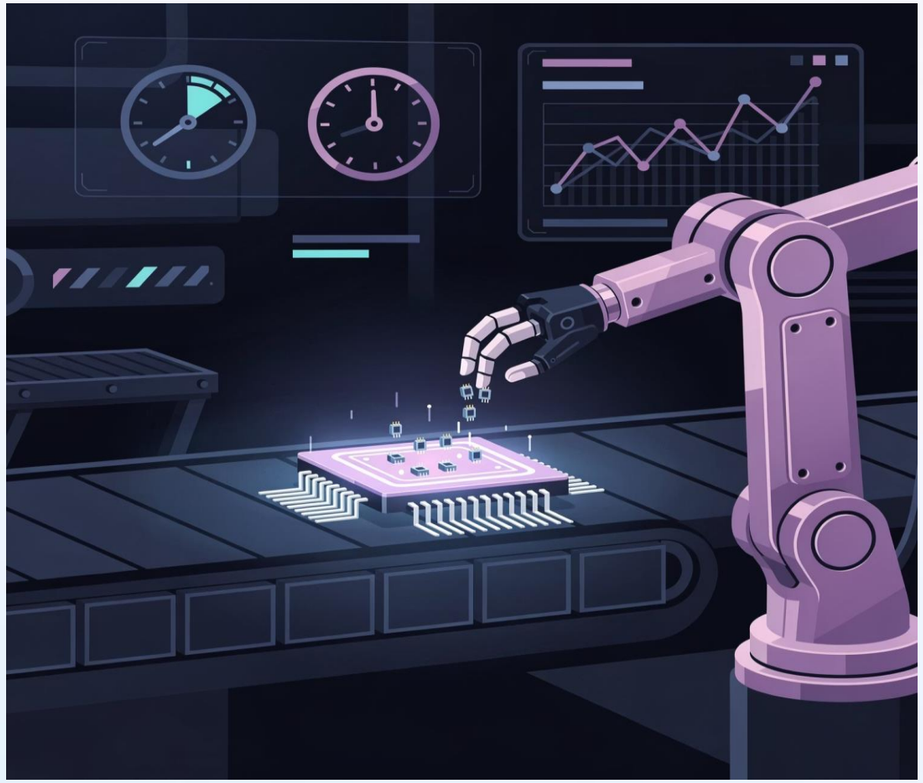
ADC/DAC: Convert between analog and digital signals

Communication: Serial, SPI, I2C, CAN, Ethernet interfaces



Critical Security Insight: Firmware stored in flash memory is a critical asset. It contains all the logic that governs device behavior, making it a prime target for attackers and requiring robust protection mechanisms.

Why Real-Time Behavior Matters



Industrial control systems must respond to inputs and generate outputs within strict, predictable time limits. This requirement fundamentally shapes how embedded systems are designed and programmed.

In OT environments, timing is not just about performance—it's about safety and reliability.

Consequences of Timing Violations

Process Instability

Control loops fail to maintain setpoints, causing oscillations, overshoot, or drift in critical process variables

Equipment Damage

Delayed protective actions allow unsafe conditions to persist, potentially damaging motors, valves, or other physical equipment

Safety Risks

Emergency shutdown systems that respond too slowly can fail to prevent hazardous situations from escalating

Security Control Requirement: Embedded security mechanisms—including cryptographic verification, access controls, and logging—must never break timing guarantees. Security features must be designed to execute within available time budgets or be placed outside critical control paths.

Introduction to RTOS

As embedded applications grow in complexity, managing multiple concurrent activities becomes challenging. A Real-Time Operating System (RTOS) provides the infrastructure to safely and predictably manage multiple tasks within timing constraints.

Task Scheduling

Determines which task runs when, based on priority and system state

Timing Control

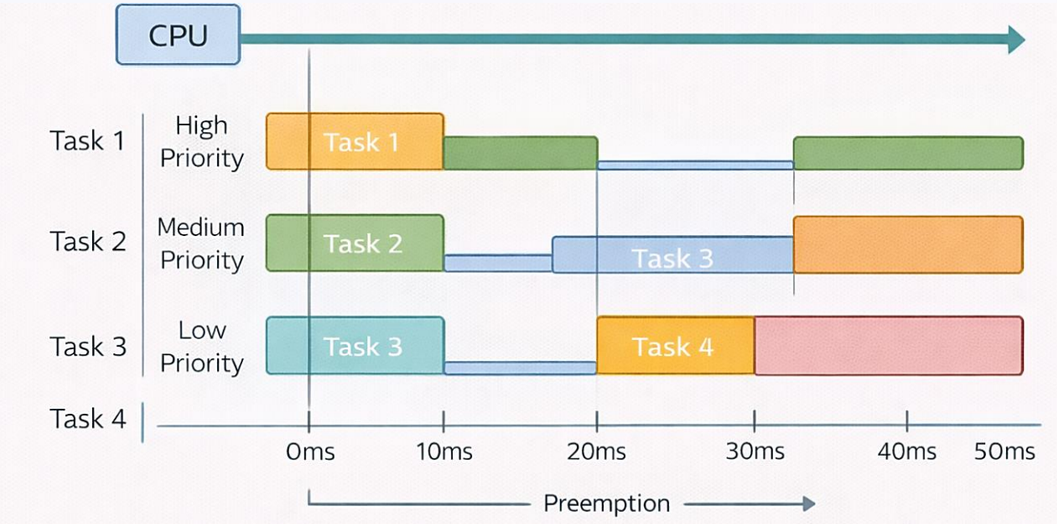
Ensures tasks meet deadlines through deterministic scheduling algorithms

Resource Management

Coordinates access to shared hardware peripherals and memory

Inter-Task Communication

Provides mechanisms for tasks to safely exchange data and synchronize



Common RTOS Applications in OT

Complex Controllers

Multi-loop controllers managing interdependent processes

Protection Systems

Safety-critical devices requiring parallel monitoring

Communication Gateways

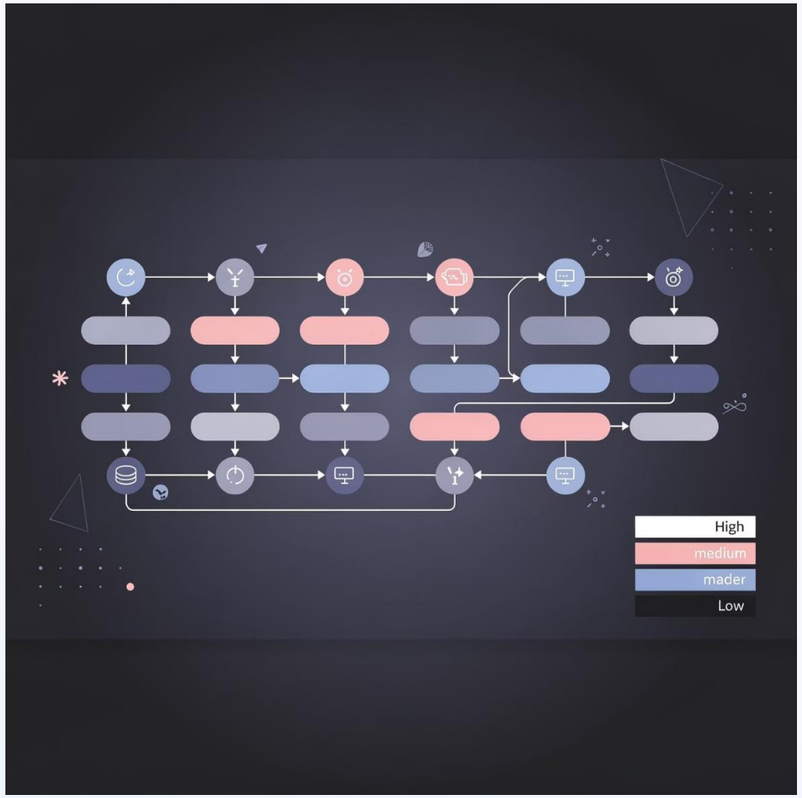
Protocol converters handling multiple simultaneous connections

Tasks, Priority & Scheduling

RTOS-based systems organize functionality into discrete tasks, each with defined priority levels and timing requirements. Understanding this structure is essential for implementing security without disrupting operations.

Task Definition

A task represents one logical function—reading a sensor, updating a control calculation, or transmitting data. Each task has its own execution context and resources.



Preemptive Scheduling

The RTOS can interrupt a running task if a higher-priority task becomes ready. This ensures critical functions execute immediately when needed, regardless of what else is running.

Priority Assignment

Priority indicates relative importance. Higher-priority tasks represent more critical functions (safety monitoring, emergency shutdown) while lower-priority tasks handle less time-sensitive activities.

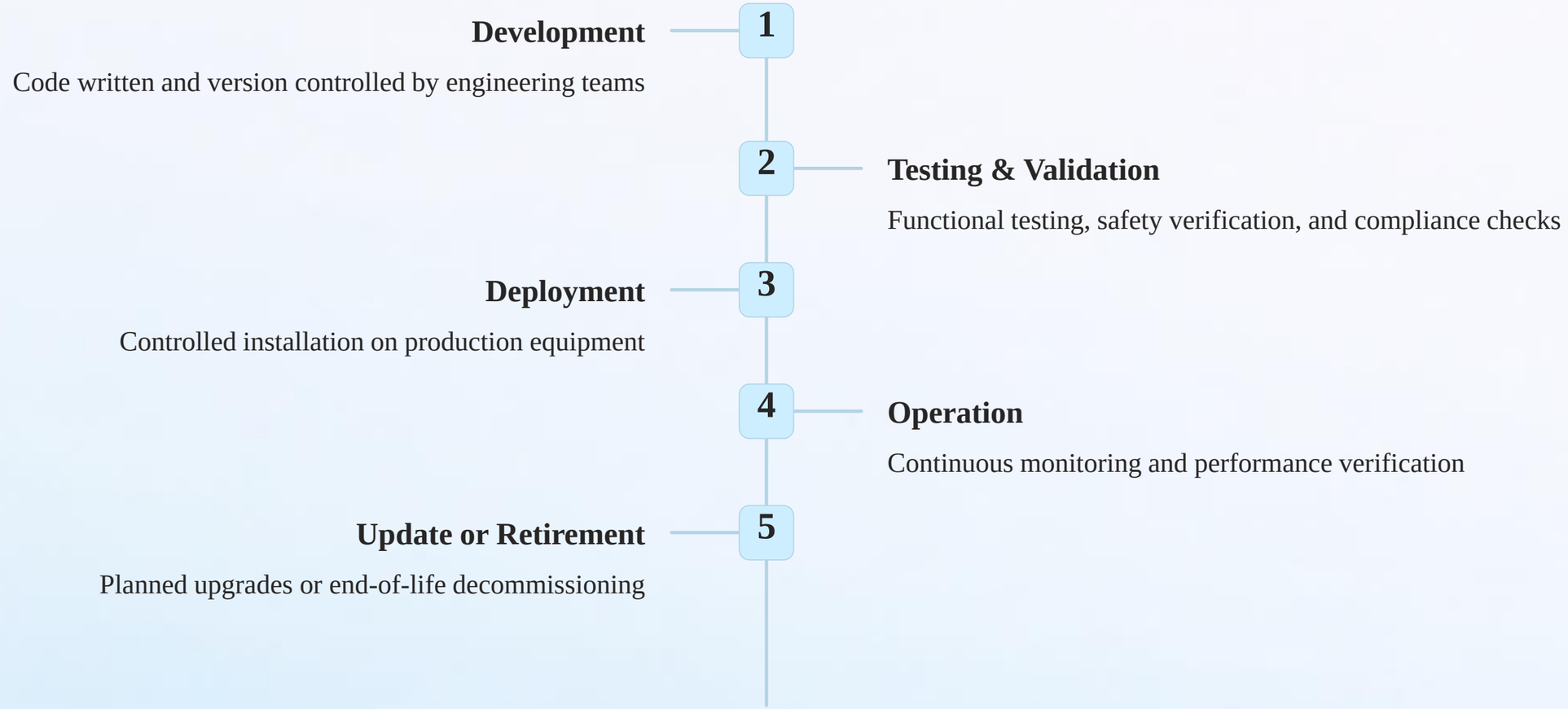
Deadline Management

Tasks must complete execution before their deadlines. The scheduler ensures sufficient CPU time is allocated to meet all timing constraints.

Security Design Principle: Security features must respect task priorities and deadlines. Authentication checks, encryption operations, and logging functions should be implemented as lower-priority tasks or executed outside time-critical control loops to avoid interfering with essential operations.

Firmware as a Controlled Asset

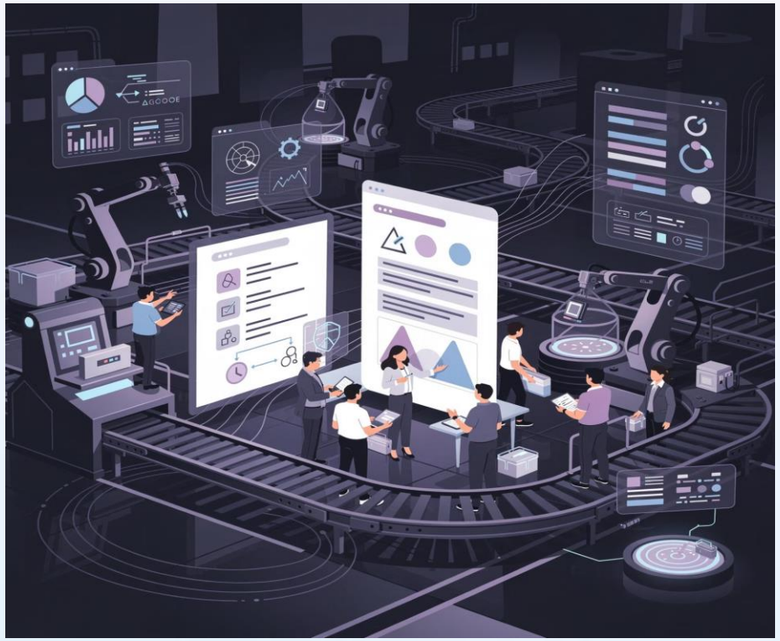
Firmware is not merely software—it is the digital embodiment of how an industrial device behaves, responds, and interacts with physical processes. This makes firmware one of the most critical assets in OT security.



Risk of Uncontrolled Changes: Uncontrolled or unauthorized firmware modifications can introduce unsafe behavior patterns, bypass safety interlocks, alter control logic, or create unpredictable interactions with other systems. Each of these outcomes poses operational and safety risks.

Firmware Governance (OT View)

In operational technology environments, firmware management follows rigorous governance processes that reflect the high stakes of industrial operations. Unlike IT software that can be rapidly patched and rolled back, OT firmware changes affect physical equipment and processes.



Planned and Approved

All firmware updates follow formal change management procedures with multiple approval stages

Thoroughly Documented

Every change is recorded with version numbers, test results, and approval signatures

Rollback Ready

Previous firmware versions are preserved and procedures exist to restore them if needed

Why Governance Matters

Physical Process Impact

Firmware directly controls motors, valves, and other equipment that manipulate physical materials and energy

Cost of Errors

Mistakes can result in production downtime, equipment damage, wasted materials, and repair costs

Safety Considerations

Incorrect firmware behavior can create hazardous conditions for personnel and the environment

Secure Boot (Concept)

Secure boot is a foundational security mechanism that establishes trust at the moment a device powers on.

It ensures that only authenticated, unmodified firmware is allowed to execute, preventing unauthorized code from controlling industrial equipment.

Device Powers On

The microcontroller begins execution from a fixed reset vector, starting the bootloader code stored in protected memory

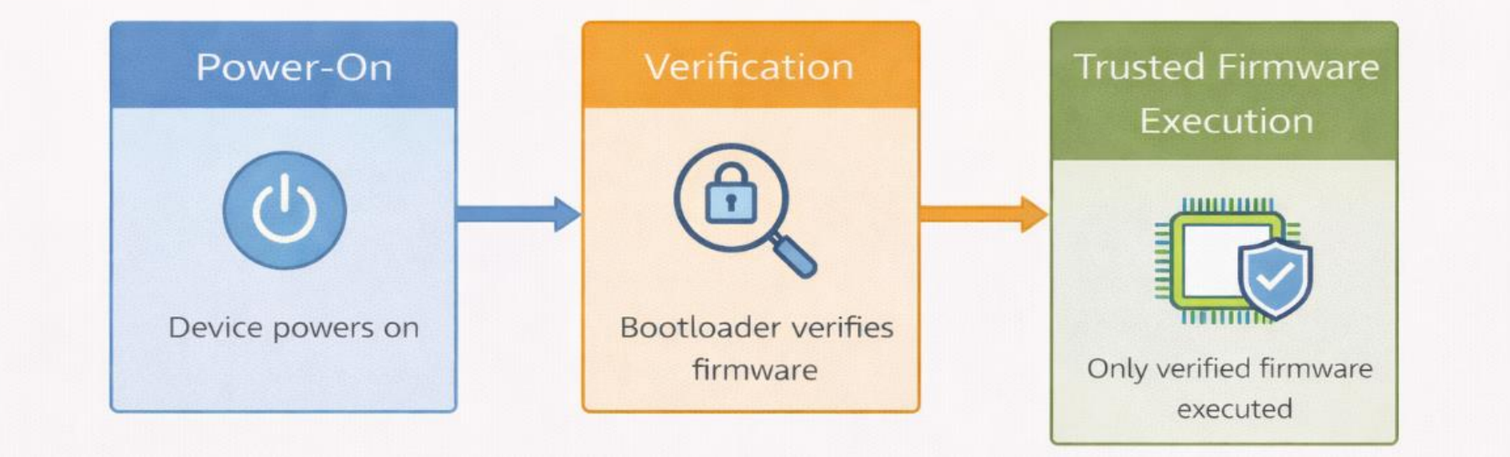
Bootloader Verification

The bootloader cryptographically verifies the firmware image using digital signatures or hash comparisons against known-good values

Conditional Execution

Only firmware that passes verification is allowed to run.

Failed verification results in the device halting or entering a safe fallback mode



Primary Goal: Prevent unauthorized, corrupted, or malicious firmware from controlling industrial equipment. Secure boot acts as the first line of defense, ensuring the device starts in a known, trusted state every time it powers on.

Why Secure Boot Is Important

Secure boot provides multiple layers of protection that align with core OT security principles: authenticity, integrity, safety, and predictability.

Authenticity

Confirms firmware originates from a trusted source—the legitimate manufacturer or authorized developer—not an attacker or unauthorized party

Integrity

Verifies firmware has not been modified, corrupted, or tampered with during storage, transmission, or after installation on the device

Safety Assurance

Prevents unsafe or unexpected startup behavior by ensuring only validated, tested firmware versions execute on production equipment

Predictability

Guarantees the same trusted behavior every boot cycle, eliminating uncertainty about device state after power cycles or failures

Educational Focus: This module concentrates on understanding *why* secure boot matters and *what* threats it mitigates, not on techniques to bypass or defeat it.

The goal is to develop security professionals who strengthen these protections.

Firmware Integrity & Digital Signatures

Digital signatures provide cryptographic proof of firmware authenticity and integrity. This technology enables secure boot implementations by creating unforgeable verification mechanisms.

How Digital Signatures Work

01

Signing During Development

The firmware developer uses a private cryptographic key to generate a unique signature for each firmware version

02

Signature Verification

Before execution, the device uses the corresponding public key to verify the signature matches the firmware

03

Rejection of Invalid Firmware

If the signature is invalid or missing, the bootloader refuses to execute the firmware

Security Outcome: Digital signatures create a cryptographic chain of trust from developer to deployed device. This mechanism prevents both accidental corruption (bit flips, storage errors) and intentional modifications (malicious tampering, unauthorized patches) from executing on industrial equipment.

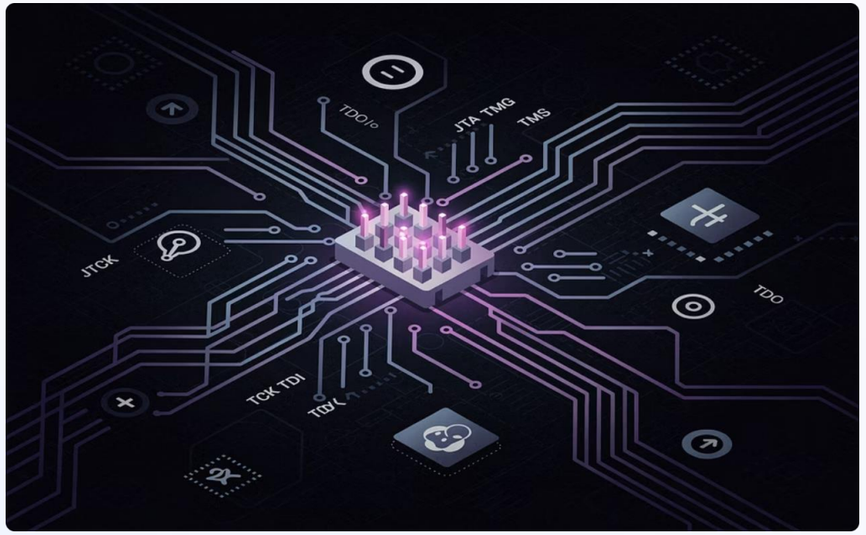
Physical Debug Interfaces

Embedded systems include hardware interfaces designed for development, debugging, and manufacturing. While essential during development, these interfaces represent significant security risks if left accessible in production deployments.

UART (Serial Port)

Purpose: Text-based logging, diagnostic output, and command-line access

Risk: Can provide root shell access, allow firmware flashing, or expose sensitive information



JTAG (Debug Port)

Purpose: Low-level hardware debugging, memory inspection, and direct CPU control

Risk: Enables complete system takeover, firmware extraction, and security bypass

Production Security Measures

- Physically remove or disable debug headers on production boards
- Implement authentication requirements for debug access
- Disable debug interfaces through configuration fuses
- Encrypt debug communications to prevent eavesdropping
- Log all debug access attempts for security monitoring

Security Principle: Debug access must be restricted or completely disabled in production environments. The convenience of debug interfaces during development becomes a critical vulnerability in deployed systems.

Secure Handling of Development Boards

Development and evaluation boards used in laboratories and training environments are fully functional embedded systems. The security practices established during learning and prototyping phases directly influence behaviors carried into professional industrial environments.

Track Board Ownership

Maintain inventory logs showing who has access to each development board and when it was issued

Restrict Flashing Rights

Implement access controls determining who can upload firmware to devices, preventing unauthorized modifications

Avoid Shared Credentials

Use unique passwords and authentication tokens for each user rather than sharing generic development credentials

Restore Boards After Use

Return boards to known-good baseline configurations between projects to prevent cross-contamination

Document All Changes

Record firmware versions, configuration changes, and hardware modifications in lab notebooks or tracking systems

Why Laboratory Practices Matter: Development boards are real embedded systems with the same capabilities as industrial devices. Security habits formed in educational and laboratory settings—good or bad—transfer directly to professional practice. Establishing rigorous security discipline early creates professionals who naturally apply these principles to production OT environments.

Module 03 Key Takeaways

This module established the foundational understanding of how embedded systems operate and why security must begin at the device level.

Embedded Systems Control Physical Processes

Industrial embedded devices directly sense, decide, and act on real-world equipment and materials

Microcontrollers Require Predictable Behavior

Deterministic, real-time response is essential for safe and reliable industrial operations

RTOS Manages Complexity Safely

Real-time operating systems coordinate multiple tasks while maintaining timing guarantees

Firmware Is a Critical Control Asset

Firmware defines device behavior and requires rigorous governance and change management

Secure Boot Establishes Trust at Startup

Cryptographic verification ensures only authorized firmware executes on industrial equipment

Physical Access Must Be Controlled

Debug interfaces and development tools provide deep system access requiring protection

Lab Practices Shape Industry Behavior

Security habits developed during training carry forward into professional practice

Reflection & Class Discussion

This module covered the foundational elements of embedded systems security. Use these discussion points to reinforce understanding and encourage critical thinking about OT security principles.

- Why microcontrollers are critical assets in OT systems
- Difference between bare-metal firmware and RTOS-based systems
- How real-time constraints affect security decisions
- Why firmware is treated as a controlled industrial asset
- Importance of restricting physical debug interfaces (UART, JTAG)

Homework & Assignments

1. Identify 5 components of an MCU and explain their role
2. Compare bare-metal vs RTOS-based systems (any 3 points)
3. Draw a basic MCU architecture diagram (CPU, memory, peripherals)
4. Explain the firmware lifecycle with one security concern per stage
5. Write the conceptual steps of secure boot in your own words

Applied Learning Task (Optional)

Lab Awareness Exercise:

- Pick one development board and list:
- Firmware storage location
- Debug interfaces available
- Two security rules to follow before deployment

COMING NEXT

What's Next? Industrial Communication Protocols

Having established how individual embedded devices operate, we now expand our view to examine how industrial systems communicate with each other.

Module 04: Industrial Communication Protocols — A Security View

You Will Learn:

Why Protocols Matter in OT

Understanding the unique security considerations of industrial communication

Modbus Communication

Polling-based protocol architecture and its security implications

MQTT Publish/Subscribe

Event-driven messaging patterns in modern IIoT environments

OPC UA Security

Service-oriented architecture with built-in security mechanisms

Protocol Monitoring

What metadata, timing, and heartbeats reveal about system behavior

Non-Disruptive Monitoring

Techniques for observing operations without interfering with control systems

Thank you for completing Module 03. Let's continue with Module 04.